TinyStress: Lightweight TinyML for Real-time Stress Detection on Wearable Devices

***Abstract*—Wearable device-enabled real-time stress detection offers valuable insights for mental health monitoring and well-being assessment. Recent studies have increasingly adopted deep neural network (NN) to this setting because of its strong representation learning capability. However, they typically requires large NNs to achieve reliable outcomes, which is infeasible on wearable devices due to computational, memory, and power limitations. This paper introduces TinyStress, an end-to-end TinyML-based stress classification model tailored for edge and wearable deployment. Our approach designs a compact CNN architecture based on depthwise separable convolutions to reduce MACs and memory footprint, enabling it to be well-suited for always-on inference under tight energy budgets. We prototyped TinyStress on an experimental testbed using an Arduino Nano 33 BLE Sense as to measure energy consumption, memory footprint, and inference latency and a MAX30105 sensor to collect human PPG data. Extensive results on WESAD dataset demonstrated that our approach achieves a 3× reduction in computational complexity compared with a standard CNN-based model, while incurring only a 3% accuracy decrease. On hardware, it also reduce flash footprint, inference time and energy usage, respectively, by up to 1.5×, 4×, and 3×, offering a cost-effective, real-time, and energy-efficient solution for continuous stress monitoring.**

***Index Terms*—Stress Detection, TinyML, Depthwise Separable Convolution.**

I. Introduction

Motivation: Stress has become a major health concern with serious consequences for both physical and mental well-being. Prolonged stress exposure is associated with cardiovascular conditions, anxiety and depression, weakened immune responses, and lower productivity, underscoring the importance of effective real-time monitoring and management [1], [2]. Conventional stress assessment has largely depended on subjective instruments, including self-report questionnaires and clinician-administered evaluations. Although these approaches can capture aspects of psychological state, they are vulnerable to reporting and observer biases, occur only intermittently, and are poorly suited for continuous, real-time monitoring [3]. Consequently, there is growing demand for objective, continuous, and noninvasive stress sensing, accelerating the development of next-generation wearable technologies.

Among wearable modalities, photoplethysmography (PPG) is widely adopted due to its easy integration into consumer devices, noninvasive sensing, low cost, and comfort for long-term use [4]. PPG measures peripheral blood volume changes, providing signals that reflect autonomic nervous system activity relevant to stress [5]. However, PPG is highly susceptible to motion artifacts, ambient light interference, and inter-subject physiological variability, requiring robust signal processing [6]. Conventional feature-engineering approaches [7] often employ numerous handcrafted descriptors or frequency-domain transformations. It not only attains limited accuracy (i.e., ∼74%) but also incurs substantial resource overhead (i.e., 1.2 MB memory), which can impede real-time deployment on resource-constrained embedded devices that are limited to 1 MB of flash storage. On the other hand, recent studies have investigated the integration of deep learning (DL) into wearable devices to enhance the performance. For instance, [8] introduced a LSTM-based approach, and [9], [10] utilized standard CNN to enhance the performance, achieving ∼85% in accuracy.

Existing Limitation: Despite this advantage, several limitations still hinder the practical deployment of DL-based wearable sensing systems. In particular, almost healthcare applications require reliable performance under real-world conditions, which often motivates the use of large and computationally intensive models. For example, [8] using LSTM-based approach require 10.2 MB of storage, while [9], [10] with 1D-CNN also demand 0.6 MB. Such models are struggle to run on wearable devices due to strict constraints on energy consumption, memory footprint, and battery life. This motivates the following research question: *"How can we design an effective DL-based model that can operate directly on resource-constrained wearable devices?"*

Key Contributions: We response this question by proposing TinyStress, an end-to-end TinyML-based model for stress detection from physiological signals that preserves high accuracy while enabling on-device deployment. Our design adopts depthwise separable convolutions (DSConv) as the backbone of the network. The core idea of DSConv is to factorize a standard convolution into two stages: i) a "*depthwise*" convolution that performs spatial filtering independently on each input channel, followed by a "*pointwise*" convolution that mixes information across channels. This decomposition substantially reduces the number of parameters and multiply–accumulate operations compared to conventional convolutions, thereby lowering model complexity and inference latency.

Summary of the main contributions:

- We propose TinyStress, a lightweight TinyML stress-detection model that utilizes DSConv as its backbone to reduce memory footprint and latency overhead while maintaining competitive accuracy. This design enables continuous, energy-efficient stress tracking for always-on wearable operation.
- We prototype TinyStress on an experimental testbed consisting of Arduino Nano 33 BLE Sense as resource-

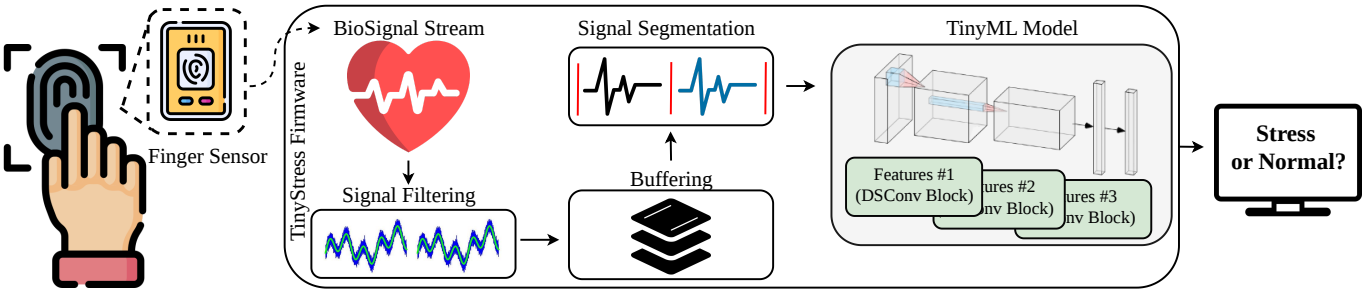

Fig. 1. End-to-End On-Device Pipeline of `TinyStress` for PPG-Based Stress Classification.

limited wearable device, to measure energy consumption, memory footprint, and inference latency. Experimental results indicate that our approach achieves a 3× reduction in computational complexity compared to standard CNN-based model, while incurring only a 3% accuracy decrease. It also reduces flash footprint, inference time, and energy usage, respectively, by up to 1.5×, 4×, and 3×, making it well-suited for efficient deployment on embedded and wearable platforms.

Paper organization: Section II reviews related work, and Section III presents the `TinyStress` framework, while Section IV evaluates its performance. Finally, Section V concludes the paper.

II. Related Work

PPG-based Stress Detection: PPG has been widely studied for stress detection because it captures stress-related hemodynamic signatures such as heart rate variability and pulse amplitude. For example, prior work [11] reported that PPG-derived features, combined with additional physiological signals, achieved roughly 90% accuracy for stress recognition in real-world driving tasks. While these studies [7]–[10] established PPG's feasibility, their reliance on computationally heavy feature extraction limits real-time deployment on wearable devices. Notably, recent work have increasingly combined PPG with additional physiological signals such as electrodermal activity or skin temperature to further improve performance. For instance, [12] integrated PPG and EDA signals to achieve stress classification accuracies exceeding 95% under controlled lab conditions. However, the computational burden grows with each added sensor modality, creating obstacles for continuous real-time use on power-constrained devices.

Deep Learning in Stress Detection: Deep learning models have recently simplified feature extraction by learning discriminative patterns directly from raw signals. For instance, [13] proposed an LSTM-based real-times tress classification system that processes raw PPG waveform low-power platforms, maintaining around 90% accuracy. However, many deep architectures remain compute-intensive, and minimizing preprocessing can degrade performance when raw signals are noisy. Therefore, TinyML approaches have been proposed to balance accuracy and efficiency. For example, [14] showed that wavelet-based features combined with a pruned CNN can maintain strong accuracy with model sizes under 2 MB. Similarly, [15] used an autoencoder for feature extraction with an edge-oriented SVM classifier, achieving nearly 95% accuracy. Nevertheless, many of these methods depend on complex hand-crafted features or transform-based preprocessing, which can hinder generalization and complicate deployment. To address these limitations, we propose a lightweight TinyML approach for stress classification that maintains high accuracy while reducing energy consumption and inference latency, enabling real-time on-device deployment on wearable platforms.

III. `TinyStress` System Overview

We design a lightweight end-to-end pipeline for classifying stress-related patterns from raw PPG signals on an embedded microcontroller. In our approach, PPG samples are streamed to the MCU over a serial link, then processed in firmware using a lightweight high-pass filter to remove baseline drift. The filtered stream is stored in a ring buffer, and a sliding-window segmentation scheme (window length N with a configurable hop size) is used to form inference windows. For each window, the firmware applies per-window normalization (mean and standard deviation) before feeding the data to a TinyML model deployed with TensorFlow Lite Micro. The DSConv-based network extracts discriminative temporal features and outputs a stress versus non-stress decision, which is reported via serial logs and displayed on the monitor or device.

A. Signal Filtering

To suppress low-frequency baseline drift in the raw PPG stream, the firmware applies a lightweight first-order high-pass filter before buffering and segmentation. Baseline drift typically arises from slow variations such as motion-induced pressure changes, respiration-related trends, and gradual sensor offset, which can obscure the pulsatile component that carries stress-relevant information. We implement the filter in a causal, sample-by-sample form with a low cutoff frequency, f_c at the system sampling rate, f_s, requiring only a few scalar state variables and constant-time computation per sample. This design introduces negligible memory overhead and is well-suited to real-time execution on resource-constrained MCUs,

while improving input stability by attenuating slow trends and preserving the higher-frequency cardiac-synchronous waveform used by the downstream TinyML classifier.

B. Buffering

To support continuous streaming and real-time inference on the MCU, we store incoming samples in a fixed-size ring buffer. A ring buffer is a circular memory structure in which new samples overwrite the oldest ones once the buffer capacity is reached, enabling constant-memory operation without dynamic allocation. In our firmware, each filtered PPG sample is pushed into the circular buffer and indexed by a write pointer, allowing the system to always access the most recent N samples required for inference. This design decouples acquisition from computation, stabilizes the pipeline under bursty serial arrivals, and enables efficient sliding-window segmentation with configurable hop sizes while minimizing the risk of sample loss or timing bottlenecks on resource-constrained microcontrollers.

C. Signal Segmentation

Signal segmentation in our system is implemented as a sliding-window scheme to convert the continuous PPG stream into fixed-length inputs required by the TinyML model. After samples are filtered and stored in the ring buffer, the firmware continuously forms an inference window of the most recent N samples and advances this window by a configurable hop size (e.g., 128–512 samples). This hop-based scheduling controls the inference rate and provides a tunable trade-off between responsiveness and computational load: smaller hops yield more frequent predictions but increase CPU utilization, while larger hops reduce workload at the cost of slower updates. By segmenting the stream online in firmware, the MCU can perform continuous monitoring and generate timely stress decisions without interrupting data acquisition.

D. TinyML Architecture

Pre-processing: Each raw PPG window $\mathbf{x} \in \mathbb{R}^T$ is reshaped into a single-channel sequence $\mathbf{X} \in \mathbb{R}^{T\times 1}$. We standardize the input using statistics computed on the training set:

$$\tilde{\mathbf{X}} = \frac{\mathbf{X} - \mu}{\sigma + \epsilon}, \tag{1}$$

where μ and σ are the training-set mean and standard deviation, respectively, and ϵ is a small constant for numerical stability. This normalization reduces inter-subject and inter-session scale variations and stabilizes training.

DSConv Block: Let $\mathbf{H}^{(0)} = \tilde{\mathbf{X}}$ denote the normalized input. The backbone consists of B convolutional blocks. The b-th block transforms $\mathbf{H}^{(b-1)} \in \mathbb{R}^{T_{b-1}\times F_{b-1}}$ to $\mathbf{H}^{(b)} \in \mathbb{R}^{T_b \times F_b}$ using a depthwise separable 1D convolution (DSConv), batch normalization, a pointwise nonlinearity, and optional max pooling.

DSConv factorizes a standard 1D convolution into a depthwise operation (channel-wise temporal filtering) and a pointwise operation (channel mixing). Specifically, the block first applies a depthwise temporal convolution with kernel size k_b independently on each input channel, producing an intermediate representation:

$$\mathbf{U}^{(b)} = \mathrm{DWConv1D}\Big(\mathbf{H}^{(b-1)}; k_b\Big), \tag{2}$$

and then applies a 1×1 pointwise convolution to mix channels and obtain F_b output channels:

$$\mathbf{Z}^{(b)} = \mathrm{PWConv1D}\Big(\mathbf{U}^{(b)}; F_b\Big). \tag{3}$$

Compared to a standard convolution with parameter cost $k_b F_{b-1} F_b$, DSConv reduces parameters to $k_b F_{b-1} + F_{b-1} F_b$, which yields substantial savings when $k_b > 1$ and F_b is moderate. To stabilize intermediate feature distributions, we apply batch normalization to $\mathbf{Z}^{(b)}$:

$$\bar{\mathbf{Z}}^{(b)} = \mathrm{BN}\Big(\mathbf{Z}^{(b)}\Big), \tag{4}$$

followed by a pointwise nonlinearity (ReLU in our implementation):

$$\mathbf{A}^{(b)} = \phi\Big(\bar{\mathbf{Z}}^{(b)}\Big). \tag{5}$$

To progressively increase the effective receptive field and reduce computational cost, we apply temporal max pooling after configurable block intervals. When pooling is enabled at block b, we obtain:

$$\mathbf{H}^{(b)} = \mathrm{MaxPool1D}\Big(\mathbf{A}^{(b)}; s\Big), \tag{6}$$

where s is the pooling size (and stride) along the temporal dimension. When pooling is disabled at block b, we set $\mathbf{H}^{(b)} = \mathbf{A}^{(b)}$. This design allows flexible control over the temporal resolution schedule while keeping the model lightweight. After the final convolutional block, we implement a fully connected layer to form a compact latent representation:

$$\mathbf{r} = \phi(\mathbf{W}_1 \tilde{\mathbf{g}} + \mathbf{b}_1), \tag{7}$$

and a final linear projection with softmax to produce class probabilities:

$$\mathbf{o} = \mathbf{W}_2 \mathbf{r} + \mathbf{b}_2,$$

$$\hat{\mathbf{y}} = \mathrm{Softmax}(\mathbf{o}). \tag{8}$$

IV. Experiment Evaluation

A. Experimental Settings

Datasets: We consider the WESAD dataset [12], which contains multi-modal physiological signals collected from 15 subjects under different emotional and physical conditions collected from both awristworn Empatica E4 device and a chest-worn RespiBAN sensor. Although WESAD dataset includes various physiological signals, we use the PPG signal was selected for analysis due to its prevalence in wearable technology and non-invasive nature. Each subject's data covers two relevant states, including 1) Baseline (Calm) State and 2) Stress State. Our experiments used the PPG signals sampled at $F_s = 64$ Hz, focusing exclusively on these two states for binary classification (stress vs. no stress).

TABLE I
STRESS DETECTION PERFORMANCE COMPARISON BETWEEN DIFFERENT SCHEMES ON SEWAD DATASET.

Methods	Accuracy (%) ↑	F1-score ↑	Confusion Matrix	Model Complexity (Kb) ↓
Standard CNN Model	89.11	87.84	2027 299 65 950	68.95
DSConv Model	85.75	81.52	2232 94 382 633	29.54
Δ	3.36	6.32	NA	39.41

System Settings: For high-pass filter, we use $f_c = 0.5$ at the system sampling rate $F_s = 64$. Following the filtering stage, the clean PPG signal was segmented using a sliding window technique to prepare fixed-length input segments suitable for machine learning models. The segmentation was performed using a sliding window approach with a window size of $N = 512$ samples, corresponding to 8 seconds at a sampling rate of $F_s = 64$ Hz. Each window overlapped the previous by 25% (128 samples), with a step size of 384 samples to advance the window. This overlapping strategy ensures that no significant transitional information between windows is lost, crucial for preserving the continuity of physiological signals and enhancing model performance.

Model Compression Methods: In this work, we apply model compression techniques, including quantization, pruning, and knowledge distillation, to analyze the trade-off between predictive performance and on-device computational complexity for `TinyStress`. Specifically, we adopt 8-bit post-training quantization due to its fast, practical deployment workflow. We further employ structured pruning to reduce computation in a hardware-friendly manner that maps well to MCU execution. For knowledge distillation, we use a 4-block CNN as the teacher network and train compact student models with 2-block and 3-block CNN backbones to retain accuracy while reducing model size and latency.

Simulation Hyperparameter: The proposed approach and baselines are implemented in PyTorch and trained on a T4-GPU (Google Colab) for 50 epochs using the Adam optimizer, with a batch size of 128, a learning rate of 0.001, and a momentum of 0.9. We use 4 CNN blocks, and the kernel sizes are 9, 7, 5, and 3, respectively.

Baselines and Evaluation Metrics: We adopt a standard 1D CNN-based model as a baseline to establish a strong yet lightweight reference point. To quantify the performance-complexity trade-off, we report classification effectiveness using accuracy, macro-F1 score and confusion matrix, and characterize model complexity using the total number of trainable parameters. In addition, we deploy both models on embedded hardware to assess practical feasibility under real device constraints, including flash storage usage and runtime memory footprint. In addition, we report CPU active time (%), defined as the fraction of the measurement window during which the microcontroller is active (i.e., not in sleep mode). We further measure on-device inference time per sample. Finally, we use an energy proxy, which approximates the total CPU work per second by aggregating computation, buffer preparation, and communication handling.

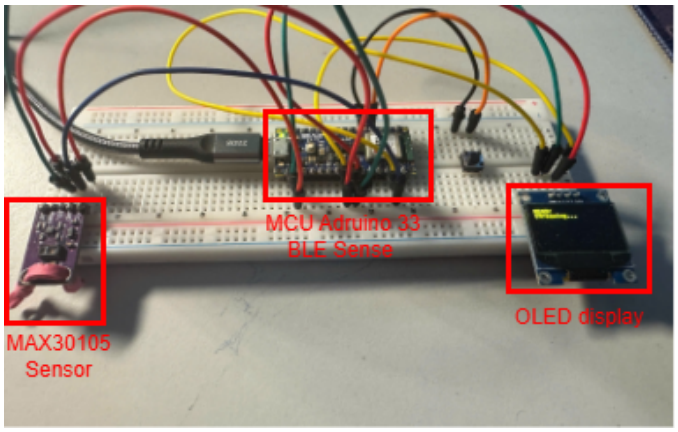

Fig. 2. Experimental setup for performance evaluation to evaluate the on-hardware resource consumption.

On-hardware Implementation: As shown in Fig. 2, we set up a scenario to evaluate resource consumption. In particular, we use a Arduino Sense 33 BLE as the resource constrained wearable device to evaluate storage requirements and runtime memory footprint. Furthermore, we integrate an OLED display and the MAX30105 sensor to support real-time operation.

B. Experimental Results

1) Accuracy and Model Complexity in Offline Simulation:

Accuracy and Macro-F1 Score: The results in Tab. I reveal a consistent accuracy-efficiency trade-off between the standard 1D CNN baseline and the proposed DSConv variant on the SEWAD dataset. The standard CNN achieves the highest predictive performance, reaching 89.11% accuracy and 87.84 macro-F1, whereas DSConv attains 85.75% accuracy and 81.52 macro-F1. This indicates that replacing full convolutions with DSConv reduces representational capacity to some extent in our current configuration, leading to a measurable drop in classification quality (3.36% in accuracy and 6.32 points in F1).

Confusion Matrix: A closer inspection of the confusion matrices suggests that the performance gap is primarily driven by increased classification of positive samples (i.e., stress) under DSConv. Specifically, DSConv produces more false negatives (382 vs. 65 for the baseline) and fewer true positives (633 vs. 950), implying reduced sensitivity to stress events. In contrast, DSConv yields fewer false positives (94 vs. 299) and more true negatives (2232 vs. 2027), which reflects a more conservative decision boundary. This asymmetric error profile is important for stress monitoring: depending on the target

TABLE II
ON-DEVICE COMPARISON ON ARDUINO NANO 33 BLE (TFLITE MICRO).

Metric	DSConv	Standard 1D CNN
Program storage (bytes)	410,392 (41%)	599,800 (61%)
Max program storage (bytes)	983,040	983,040
Global variables (bytes)	154,920 (59%)	154,928 (59%)
Free dynamic memory (bytes)	107,224	107,216
Max dynamic memory (bytes)	262,144	262,144
CPU active (%)	23.8	35.6
Inference time (ms)	50.678	192.957
Energy proxy (ms/s)	139.965	431.781

application, false negatives may be costlier than false positives (e.g., missed stress episodes in continuous monitoring), and thus model selection should be guided by the operational preference for sensitivity versus specificity.

Model Complexity: Despite the drop in offline performance, DSConv offers a substantial reduction in model complexity, decreasing the model size from 68.95 KB to 29.54 KB. This corresponds to a 39.41 KB reduction, or approximately 57% relative compression. Such savings directly translate to lower flash usage and typically lower compute requirements, which are critical constraints for wearable deployment. In scenarios where memory budget or energy consumption is the primary bottleneck, DSConv provides a favorable operating point that enables on-device inference with reduced storage demands, potentially allowing additional signal processing, robustness modules, or multi-task extensions within the same hardware envelope.

Discussion: Overall, Tab. I indicates that DSConv is an effective efficiency-oriented alternative to the standard CNN, but its current instantiation trades away sensitivity to stress. This motivates two practical follow-up directions: first, tuning DSConv capacity (e.g., modestly increasing channel widths in later blocks or adjusting kernel schedules) to recover F1 while maintaining a small footprint; and second, optimizing decision thresholds or incorporating cost-sensitive training to better control the false-negative rate in deployment settings.

2) Resource Consumption on Hardware:

Flash Footprint: The on-device results in Tab. II highlight a clear efficiency advantage of the proposed DSConv backbone over the standard 1D CNN baseline under identical deployment constraints. First, DSConv substantially reduces flash footprint, requiring 410.392 bytes, i.e., around 41% of available program storage, compared to 599.800 bytes (61%) for the baseline. This $\sim$31.6% reduction is practically important for microcontroller deployments where program memory is often the primary bottleneck, and it enables additional headroom for sensor I/O, signal processing, and application logic.

Memory Usage: These two models exhibit nearly identical dynamic memory usage (global variables ($\approx$ 155) KB, leaving ($\approx$ 107) KB for local allocations), indicating that RAM consumption is dominated by shared runtime components (e.g., tensor arena, buffering, and system code) rather than model parameters alone. Consequently, while DSConv reduces parameter count and flash storage, it does not necessarily reduce peak RAM in the current implementation. This suggests that further RAM savings would likely require reducing intermediate activation sizes (e.g., fewer channels, more aggressive pooling) or employing quantized inference with smaller tensor buffers.

CPU Active: DSConv yields a meaningful reduction in compute-related cost at runtime. The CPU active fraction drops from 35.6% to 23.8%, and the measured inference workload decreases from 192.957 to 50.678 ms per second, corresponding to a $\sim$3.8$\times$ reduction in time spent inside the inference kernel. Consistently, the overall energy proxy (which aggregates inference, input filling, and serial processing time) decreases from 431.781 to 139.965 ms per second ($\sim$3.1$\times$). These results indicate that DSConv improves the end-to-end efficiency not only at the model level but also at the system level, reducing the fraction of time the MCU remains active and therefore improving energy efficiency in streaming operation.

Inference Time: The reduction in inference cost does not translate to proportional RAM savings, and the absolute inference burden remains non-trivial under float32 TFLite Micro execution. This behavior is consistent with microcontroller inference pipelines where runtime buffers and operator implementations can dominate resource usage. In future iterations, enabling integer quantization (e.g., full int8) and optimized kernels would be expected to further reduce latency and energy, potentially unlocking higher inference rates or lower duty cycles for continuous sensing scenarios.

3) `TinyStress` using Model Compression:

Tab. III highlights clear accuracy–efficiency trade-offs when deploying `TinyStress` on MCU hardware. The baseline model achieves the best accuracy, 85.19%, but incurs the largest footprint, 227.164 bytes, and the highest per-window latency ($\sim$ 1.18 s). Applying 8-bit post-training quantization provides the most practical "first-step" compression, reducing the `.tflite` size by about 3$\times$ (to 75,224 bytes) and cutting inference time to $\sim$ 0.36 s, with only a modest accuracy drop to 82.07%. Structured pruning alone yields moderate savings at 20% pruning (83.05% accuracy, $\sim$ 0.81 s), but aggressive pruning substantially degrades performance, dropping to 70.44% at 50% pruning and to below 40% at 70% pruning, indicating that excessive channel removal eliminates discriminative temporal features in PPG. Combining pruning with quantization consistently improves deployment efficiency, achieving the fastest configurations (down to $\sim$ 0.06–0.12 s) and the smallest models, but at the cost of larger accuracy losses when pruning becomes severe.

Knowledge distillation offers a complementary compression path with better accuracy retention under smaller backbones. The distilled 3-block student preserves competitive accuracy, 82.16%, while being substantially smaller than the baseline, 76.676 bytes, and faster ($\sim$ 0.72 s), demonstrating that the teacher can transfer informative decision boundaries to compact students. When combined with 8-bit quantization, the 3-block student further reduces runtime ($\sim$ 0.23 s) with acceptable accuracy (78.62%). The 2-block student is the

TABLE III
ON-DEVICE RESULTS FOR COMPRESSION VARIANTS OF `TinyStress`.

Model	Acc	`.tflite` byte size (flash/storage)	Flash (bytes)	Inference/window (seconds)
Baseline (3 CNN blocks)	85.75%	227164	591168	~ 1.18
Baseline + 8 bit-Quant	82.07%	75224	439688	~ 0.36
Pruning 20%	83.05%	153108	517568	~ 0.81
Pruning 20% + 8 bit-Quant	78.35%	55584	420048	~ 0.27
Pruning 50%	70.44%	65456	429920	~ 0.35
Pruning 50% + 8 bit-Quant	70.27%	30608	395072	~ 0.12
Pruning 70%	39.54%	31572	396032	~ 0.16
Pruning 70% + 8 bit-Quant	36.61%	20232	384696	~ 0.06
Distillated Model (3 CNN blocks)	82.16%	76676	441136	~ 0.72
Distillated Model (3 CNN blocks) + 8 bit-Quant	78.62%	30856	395320	~ 0.23
Distillated Model (2 CNN blocks)	72.14%	21036	285496	~ 0.31
Distillated Model (2 CNN blocks) + 8 bit-Quant	72.12%	12800	377264	~ 0.11

smallest among distilled models and achieves the lowest storage (down to 12,800 bytes with quantization), but accuracy saturates around 72%, suggesting under-capacity for capturing stress-relevant morphology. Overall, the results suggest that 8-bit quantization and distillation to a 3-block student provide the best balance between accuracy and on-device efficiency, whereas heavy pruning is only suitable when strict latency and memory constraints dominate over classification performance.

V. CONCLUSION

This paper presented `TinyStress` , a lightweight TinyML stress-detection model designed for continuous, on-device inference on resource-constrained wearables. `TinyStress` adopts a depthwise separable 1D CNN backbone to reduce computational and storage demands while maintaining competitive classification performance. Experiments on WESAD demonstrate that `TinyStress` achieves a favorable accuracy–efficiency trade-off, delivering approximately a 3× reduction in computational complexity with only a modest drop in accuracy relative to a standard 1D CNN baseline. On an Arduino Nano 33 BLE Sense (TFLite Micro), `TinyStress` further translates these algorithmic savings into system-level benefits, reducing flash footprint, inference workload, and the overall energy proxy substantially compared with the baseline, enabling more practical always-on operation under tight energy budgets.

In future work, we plan to explore neuromorphic computing to further reduce on-device energy consumption. Achieving this goal will require joint hardware–software co-design, including neuromorphic-friendly model architectures and deployment toolchains, which remains a challenging but promising direction for embedded physiological sensing.

REFERENCES

[1] B. S. Mcewen, "Stress, adaptation, and disease: Allostasis and allostatic load," *Annals of the New York Academy of Sciences*, vol. 840, 1998. [Online]. Available: https://api.semanticscholar.org/CorpusID:20043016

[2] N. Schneiderman, G. Ironson, and S. D. Siegel, "Stress and health: psychological, behavioral, and biological determinants." *Annual review of clinical psychology*, vol. 1, pp. 607–28, 2005. [Online]. Available: https://api.semanticscholar.org/CorpusID:889602

[3] W. R. Lovallo, *Stress and Health: Biological and Psychological Interactions*, 3rd ed. SAGE Publications, 2016.

[4] K. B. Kim and H. J. Baek, "Photoplethysmography in wearable devices: A comprehensive review of technological advances, current challenges, and future directions," *Electronics*, vol. 12, no. 13, 2023. [Online]. Available: https://www.mdpi.com/2079-9292/12/13/2923

[5] G. G. Berntson and J. T. Cacioppo, "Heart rate variability: Stress and psychiatric conditions," 2007. [Online]. Available: https://api.semanticscholar.org/CorpusID:17034912

[6] J. Allen, "Photoplethysmography and its application in clinical physiological measurement," *Physiological Measurement*, vol. 28, no. 3, pp. R1–R39, Mar. 2007.

[7] A. Abu-Samah, D. Ghaffa, N. F. Abdullah, N. Kamal, R. Nordin, J. C. Dela Cruz, G. V. Magwili, and R. J. Mercado, "Deployment of tinyml-based stress classification using computational constrained health wearable," *Electronics*, vol. 14, no. 4, 2025. [Online]. Available: https://www.mdpi.com/2079-9292/14/4/687

[8] A. Rostami, B. Tarvirdizadeh, K. Alipour, and M. Ghamari, "Real-time stress detection from raw noisy ppg signals using lstm model leveraging tinyml," *Arabian Journal for Science and Engineering*, vol. 50, no. 10, pp. 6959–6981, 2025.

[9] P. Ganesan, Y. R. Thota, H. Shehata, and T. Nikoubin, "Tinyml based stress detection utilizing ppg signals: A lightweight approach for smart wearable devices," in *Proceedings of the Great Lakes Symposium on VLSI 2025*, ser. GLSVLSI '25. New York, NY, USA: Association for Computing Machinery, 2025, p. 941–946. [Online]. Available: https://doi.org/10.1145/3716368.3735274

[10] P. Srivastava, N. Shah, and K. Jaiswal, "Microcontroller-based edgeml: Health monitoring for stress and sleep via hrv," *Engineering Proceedings*, vol. 78, no. 1, 2024. [Online]. Available: https://www.mdpi.com/2673-4591/78/1/3

[11] J. Healey and R. Picard, "Detecting stress during real-world driving tasks using physiological sensors," *IEEE Transactions on Intelligent Transportation Systems*, vol. 6, no. 2, pp. 156–166, 2005.

[12] A. Ahmed Al Dossary, M. Chollet, and A. Vinciarelli, *From Speech and PPG to EDA: Stress Detection Based on Cross-Modal Fine-Tuning of Foundation Models*. New York, NY, USA: Association for Computing Machinery, 2025, p. 87–95. [Online]. Available: https://doi.org/10.1145/3716553.3750753

[13] A. Rostami, B. Tarvirdizadeh, K. Alipour, and M. Ghamari, "Real-time stress detection from raw noisy ppg signals using lstm model leveraging tinyml," *Arabian Journal for Science and Engineering*, vol. 50, 05 2024.

[14] Kayadibi and O. Uslu, "A lightweight st-cnn architecture based on deep learning for stress level detection from human physical activities," *Scientific Reports*, vol. 15, 09 2025.

[15] S. Mukherjee, M. S. Hossain, and G. Muhammad, "A hybrid autoencoder-svm framework for wearable stress detection: Toward edge-based tinyml solutions," *Sensors*, vol. 22, no. 4, pp. 1–16, 2022.